DMI-ST. JOHN THE BAPTIST UNIVERSTY

MANGOCHI CAMPUS

PROGRAM NAME : OPERATING SYSTEM

MODULE NAME : 351 CS 2104

STUDENT NAME : SHADRECK NKHOPE

REG NO. : 25311351019

YEAR : II

SEMESTER : III

LECTURER NAME : CRUSADE CHIWALO

ASSIGNMENT TITLE : OPERATING SYSTEM SIMULATION

DUE DATE : 28TH MAY, 2026

Introduction

EDUOS is a simplified operating system simulation project developed as part of the CS 2104 Operating Systems coursework. The purpose of the project is to demonstrate practical implementation of core operating system concepts using both the C programming language and Python.

The project simulates several fundamental OS components including process management, thread management, inter-process communication (IPC), and CPU scheduling algorithms. In addition, a Python-based scheduling simulator was developed to visualize scheduling performance and generate analytical metrics such as waiting time, turnaround time, and response time.

The system combines low-level system simulation in C with higher-level analysis and visualization in Python, creating a hybrid educational operating system simulator.

Project Objectives

The main objectives of the project were:

- To simulate Process Control Blocks (PCBs)
- To implement process lifecycle management
- To simulate thread pools and concurrent execution
- To implement inter-process communication mechanisms
- To implement CPU scheduling algorithms
- To generate scheduling metrics and Gantt charts
- To integrate C and Python components into one workflow
- To demonstrate understanding of operating system concepts

System Overview

The EDUOS project is divided into two major sections:

C Core System

The C component simulates low-level operating system functionality such as:

- Process management
- Thread management
- IPC systems
- CPU scheduling
- PCB generation

Python Scheduling Simulator

The Python component is responsible for:

- Scheduling analysis
- Metrics computation
- Gantt chart generation
- CSV/JSON workload processing
- Visualization of scheduling behavior

Project Structure

```
EDUos/
|
├── c_core/
|   ├── main_sim.c
|   ├── process_manager.c
|   ├── thread_manager.c
|   ├── ipc_module.c
|   ├── scheduler.c
|   ├── include/
|   |   └── eduos.h
|   ├── Makefile
|   └── eduos.exe
|
├── python_scheduler/
|   ├── scheduler_sim.py
|   ├── sample_processes.csv
|   ├── pcb_snapshot.json
|   ├── requirements.txt
|   └── __pycache__/
|
├── docs/
|   └── screenshots/
|
├── README.md
└── .gitignore
```

C Core System

Process Management

The process management module simulates Process Control Blocks (PCBs). Each PCB contains information such as:

- Process ID (PID)
- Process state
- Scheduling information
- Priority information

The simulator supports process state transitions including:

- READY
- RUNNING
- TERMINATED

The system also exports PCB snapshots into JSON format for analysis by the Python scheduler.

Thread Management

The thread management module implements a simulated thread pool. Threads are responsible for executing scheduled tasks concurrently.

Features implemented include:

- Thread pool initialization
- Task queue handling
- Concurrent task execution
- Thread shutdown and cleanup

The simulator demonstrates many-to-one style thread management concepts.

Inter-Process Communication (IPC)

The IPC module simulates communication between processes using:

- Pipe-based communication
- Shared memory simulation
- Message passing

The IPC system demonstrates how processes exchange information during execution.

Python Scheduling Simulator

The Python scheduling simulator was developed to analyze CPU scheduling behavior.

Features implemented include:

- Process workload loading from CSV files
- PCB snapshot loading from JSON
- Scheduling metrics computation
- Gantt chart generation
- Algorithm comparison charts

The simulator computes:

- Waiting Time (WT)
- Turnaround Time (TAT)
- Response Time (RT)

The simulator also generates visual scheduling outputs using Matplotlib.

Scheduling Algorithms Implemented

1. FCFS (First Come First Serve)

Processes are executed in order of arrival time.

Characteristics:

- Simple implementation
- Non-preemptive
- Can cause convoy effect

2. SJF (Shortest Job First)

Processes with the shortest burst time execute first.

Characteristics:

- Minimizes average waiting time
- Non-preemptive
- May cause starvation

3. Priority Scheduling

Processes are scheduled according to priority values.

Features:

- Lower priority number indicates higher priority
- Aging mechanism implemented
- Reduces starvation

4. Round Robin

Processes execute using fixed time quanta.

Features:

- Preemptive scheduling
- Fair CPU allocation
- Improved response time

Thread Management

The simulator includes thread scheduling functionality in Python.

Features:

- Thread process grouping
- Context switching simulation
- Thread Round Robin scheduling
- Context switch overhead tracking

Inter-Process Communication (IPC)

The IPC module demonstrates process communication using simulated operating system techniques.

Implemented mechanisms include:

Pipe Communication

- Allows message transfer between processes.

Shared Memory

- Simulates memory sharing between processes.

Build and Execution Process

Building the C Core

Compilation command:

```
gcc -Wall -Wextra -pthread -std=c11 \
main_sim.c process_manager.c thread_manager.c ipc_module.c scheduler.c \
-o eduos
```

Execution:

```
./eduos
```

Windows:

eduos.exe

Running the Python Schedule

Execution command:

python scheduler_sim.py --file sample_processes.csv

This command:

- Loads scheduling data
- Runs all algorithms
- Computes metrics
- Generates Gantt charts

Challenges Encountered and Solutions

Multiple main() Function Conflict

Problem

Compilation failed because multiple source files contained main() functions.

Solution

Only main_sim.c was compiled as the primary executable entry point.

Windows Valgrind Limitation

Problem

Valgrind is not natively supported on Windows PowerShell.

Solution

MSYS2/WSL environments were recommended, and limitations were documented.

Race Conditions

Problem

Concurrent scheduling operations caused synchronization issues.

Solution

Mutex-based synchronization mechanisms were implemented.

IPC Communication Errors

Problem

Pipe and shared memory communication occasionally failed.

Solution

Additional validation and error handling were added.

Build Errors

Problem

Wildcard compilation caused linker conflicts.

Solution

Explicit source file compilation was used.

Conclusion

The EDUOS project successfully demonstrated major operating system concepts through a hybrid implementation using C and Python.

The project implemented:

- Process management
- Thread management
- IPC systems
- CPU scheduling algorithms
- Scheduling visualization and metrics analysis

The simulator provided practical understanding of operating system internals and scheduling behavior while also demonstrating software engineering practices such as modular design, build automation, and debugging.

References

Silberschatz, Galvin & Gagne – Operating System Concepts

Stallings – Operating Systems: Internals and Design Principles

GCC Documentation

https://gcc.gnu.org/onlinedocs/

POSIX Threads Documentation

https://man7.org/linux/

Python Documentation

https://docs.python.org/3/

MSYS2 Documentation

https://www.msys2.org/

351 CS 2104 Operating Systems Lecture Notes